

Data Handling and File Handling in Python

Contents

1	Introduction	3
2	Data Handling in Python	3
2.1	Basic Data Types	3
2.2	Example	3
3	Accessing Data	4
3.1	Accessing List Elements	4
3.2	Accessing Dictionary Values	4
4	Updating Data	4
5	Deleting Data	4
6	Looping Through Data	4
7	Sorting Data	5
8	Filtering Data	5
9	Mapping Data	5
10	File Handling in Python	5
11	File Modes	6
12	Writing to a File	6
13	Reading a File	6
14	Reading Line by Line	6
15	Reading All Lines	6
16	Appending Data	7
17	Using With Statement	7
18	Important File Methods	7

19 seek() and tell()	7
20 Checking File Exists	8
21 Working with CSV Files	8
21.1 Writing CSV	8
21.2 Reading CSV	8
22 Working with JSON	8
22.1 Writing JSON	8
22.2 Reading JSON	9
23 Exception Handling	9
24 Mini Project: Student Record System	9
24.1 Adding Records	9
24.2 Displaying Records	10
25 Difference Between Data Handling and File Handling	10
26 Summary	10

1 Introduction

Python provides powerful features for managing data and files efficiently.

Data Handling refers to storing, manipulating, filtering, sorting, and processing data during program execution.

File Handling refers to storing data permanently in files and retrieving it whenever required.

Both concepts are essential in data science, machine learning, software development, and automation.

2 Data Handling in Python

Data handling means performing operations on different types of data.

Python supports multiple built-in data types.

2.1 Basic Data Types

- Integer
- Float
- String
- Boolean
- List
- Tuple
- Set
- Dictionary

2.2 Example

```
1 name = "John"
2 age = 22
3 salary = 50000.75
4 is_student = True
5
6 marks = [85, 90, 78]
7
8 coordinates = (20, 50)
9
10 cities = {"Delhi", "Mumbai", "Pune"}
11
12 student = {
13     "Name": "John",
14     "Age": 22,
15     "Course": "Python"
16 }
```

3 Accessing Data

3.1 Accessing List Elements

```
1 numbers = [10,20,30,40,50]
2
3 print(numbers[0])
4 print(numbers[-1])
```

Output

10
50

3.2 Accessing Dictionary Values

```
1 student = {
2     "name": "John",
3     "age": 21
4 }
5
6 print(student["name"])
7 print(student.get("age"))
```

4 Updating Data

```
1 student = {
2     "name": "John",
3     "age": 21
4 }
5
6 student["age"] = 22
7 student["city"] = "Delhi"
8
9 print(student)
```

5 Deleting Data

```
1 student.pop("age")
2 print(student)
```

6 Looping Through Data

```

1 students = [
2     {"name": "A", "marks": 90},
3     {"name": "B", "marks": 80},
4     {"name": "C", "marks": 75}
5 ]
6
7 for s in students:
8     print(s["name"], s["marks"])

```

7 Sorting Data

```

1 numbers = [8,4,2,9,1]
2
3 numbers.sort()
4
5 print(numbers)

```

Reverse Sorting

```

1 numbers.sort(reverse=True)
2 print(numbers)

```

8 Filtering Data

```

1 numbers = [10,15,20,25,30]
2
3 even = list(filter(lambda x:x%2==0,numbers))
4
5 print(even)

```

9 Mapping Data

```

1 numbers = [1,2,3,4]
2
3 square = list(map(lambda x:x*x,numbers))
4
5 print(square)

```

10 File Handling in Python

File handling allows data to be stored permanently.

Python uses the **open()** function.

Syntax

```
open(filename, mode)
```

11 File Modes

Mode	Description
r	Read only
w	Write (overwrite file)
a	Append data
x	Create new file
r+	Read and Write
w+	Write then Read
rb	Read binary file
wb	Write binary file

12 Writing to a File

```
1 file = open("data.txt", "w")
2
3 file.write("Hello Python")
4
5 file.close()
```

13 Reading a File

```
1 file = open("data.txt", "r")
2
3 print(file.read())
4
5 file.close()
```

14 Reading Line by Line

```
1 file = open("data.txt", "r")
2
3 for line in file:
4     print(line.strip())
5
6 file.close()
```

15 Reading All Lines

```
1 file = open("data.txt", "r")
2
3 lines = file.readlines()
```

```

4
5 print(lines)
6
7 file.close()

```

16 Appending Data

```

1 file = open("students.txt","a")
2
3 file.write("David\n")
4
5 file.close()

```

17 Using With Statement

The **with** statement automatically closes the file.

```

1 with open("data.txt","r") as file:
2     print(file.read())

```

Writing Example

```

1 with open("data.txt","w") as file:
2     file.write("Python Programming")

```

18 Important File Methods

Method	Description
read()	Read complete file
readline()	Read one line
readlines()	Read all lines as list
write()	Write text
writelines()	Write multiple lines
seek()	Move file pointer
tell()	Current file position
close()	Close file

19 seek() and tell()

```

1 with open("data.txt","r") as file:
2
3     print(file.tell())
4
5     print(file.read(5))
6

```

```
7     print(file.tell())
8
9     file.seek(0)
10
11    print(file.read())
```

20 Checking File Exists

```
1 import os
2
3 if os.path.exists("data.txt"):
4     print("File Exists")
5 else:
6     print("File Not Found")
```

21 Working with CSV Files

21.1 Writing CSV

```
1 import csv
2
3 with open("students.csv","w",newline="") as file:
4
5     writer = csv.writer(file)
6
7     writer.writerow(["Name","Marks"])
8     writer.writerow(["John",90])
9     writer.writerow(["Alice",95])
```

21.2 Reading CSV

```
1 import csv
2
3 with open("students.csv","r") as file:
4
5     reader = csv.reader(file)
6
7     for row in reader:
8         print(row)
```

22 Working with JSON

22.1 Writing JSON

```

1 import json
2
3 student = {
4     "name": "John",
5     "age": 22,
6     "course": "Python"
7 }
8
9 with open("student.json", "w") as file:
10     json.dump(student, file, indent=4)

```

22.2 Reading JSON

```

1 import json
2
3 with open("student.json", "r") as file:
4     data = json.load(file)
5
6 print(data)

```

23 Exception Handling

```

1 try:
2
3     with open("demo.txt", "r") as file:
4         print(file.read())
5
6 except FileNotFoundError:
7     print("File not found")
8
9 except Exception as e:
10    print(e)

```

24 Mini Project: Student Record System

24.1 Adding Records

```

1 with open("students.txt", "a") as file:
2
3     name = input("Enter Name: ")
4     marks = input("Enter Marks: ")
5
6     file.write(f"{name},{marks}\n")
7
8 print("Record Saved")

```

24.2 Displaying Records

```
1 with open("students.txt","r") as file:
2
3     for line in file:
4         name, marks = line.strip().split(",")
5
6         print("Name:",name)
7         print("Marks:",marks)
8         print("-----")
```

25 Difference Between Data Handling and File Handling

Data Handling	File Handling
Manipulates data in memory	Stores data permanently
Uses variables, lists, tuples, dictionaries	Uses files like txt, csv, json
Temporary data	Permanent storage
Used for processing and analysis	Used for reading and writing data

26 Summary

- Data handling involves storing, processing, filtering, sorting, and manipulating data.
- File handling is used to read and write data from files.
- The **with** statement is recommended because it automatically closes the file.
- Python provides built-in modules such as **csv**, **json**, and **os** for efficient file operations.
- Exception handling makes file operations safer by preventing program crashes.